אוניברסיטת
אריאל
בשומרון

Slide set # 9
Object-Oriented Programming (2)

Introduction to Programming for Engineers

Dr. Inon Zuckerman, Dr. Chen Hajaj

Industrial Engineering and Management

חלק מהשקפים נלקחו מקורס דומה באוניברסיטת תל אביב ולהם התודה.

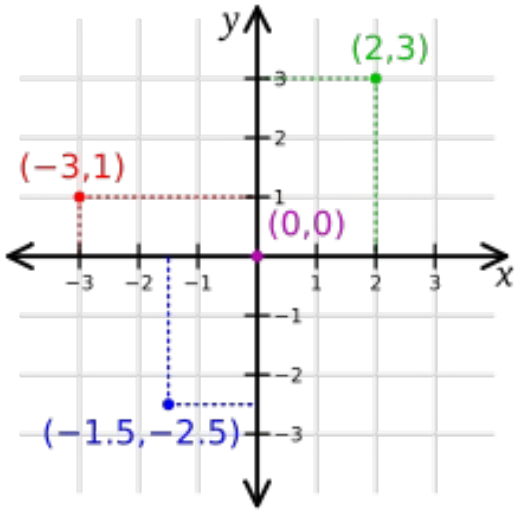
Last week - highlights

- **Object Oriented Programming**
  - •Enables representing real world entities by adding new data types – do this by defining new **Classes**.
  - •Each **class** holds both data (**attributes**) and functions (**methods**) of the entity it represents.
  - •**Classes** are blueprints for creating **Objects.**
  - •**Instantiation** is the process of creating new objects (memory allocation) from a certain class.
    - •Example: the string class is a predefined python class from which we create distinct string objects every time we create a string variable.

•**IMAGINE !** What data types would YOU need in your future programs ?

Representing a **Point** in OOP

- Attributes (Fields/Variables/Properties):
  - x
  - y
- Methods (Functions):
  - **constructor**
  - distance
  - print_point



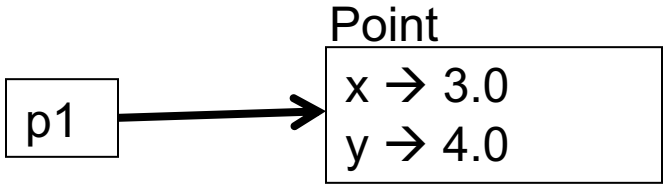
The **Point** Class

אוניברסיטת אריאל בשומרון

```
from math import sqrt

class Point:
    """represents a point in a 2D space.
    Attributes: x, y """

    def __init__(self, x, y):
        self.x = x
        self.y = y

    def print_point(point):
        print('<', point.x, ',', point.y, '>')

    def distance(self, other):
        return sqrt((self.x-other.x)**2 + (self.y-other.y)**2)
```

Attributes hold the object's data

```
>>> p1 = Point(3.0, 4.0)
>>> p1.x
3.0
>>> p1.y
4.0
```



- The variable *p1* refers to a Point object
- p1.x means "get the value of *x* from object *p1*"

Methods implements the object's functionality

אוניברסיטת אריאל בשומרון

Method

a function that is associated with a particular class.

Examples in strings, lists, dictionaries, tuples:

list.append(), str.upper(), dict.items()

Difference between methods and functions:

- Methods are defined inside a class definition
- The syntax for invoking a method is different:
  - A method is called through an instance

Example: Distance Between Two Points (method vs. function)


```
from math import sqrt

class Point:
    .......
    def distance(self, other):
        return sqrt((self.x-other.x)**2 + (self.y-other.y)**2)
    .......

def distance(p1, p2):
    return sqrt((p1.x - p2.x)**2 + (p1.y - p2.y)**2)
```

```
>>> p1 = Point(2,-3)
>>> p2 = Point(4,5)
>>> p1.distance(p2)
8.246211251235321
>>> distance(p1,p2)
8.246211251235321
```

Method call

function call

Adding a **Circle** class which uses **Point**

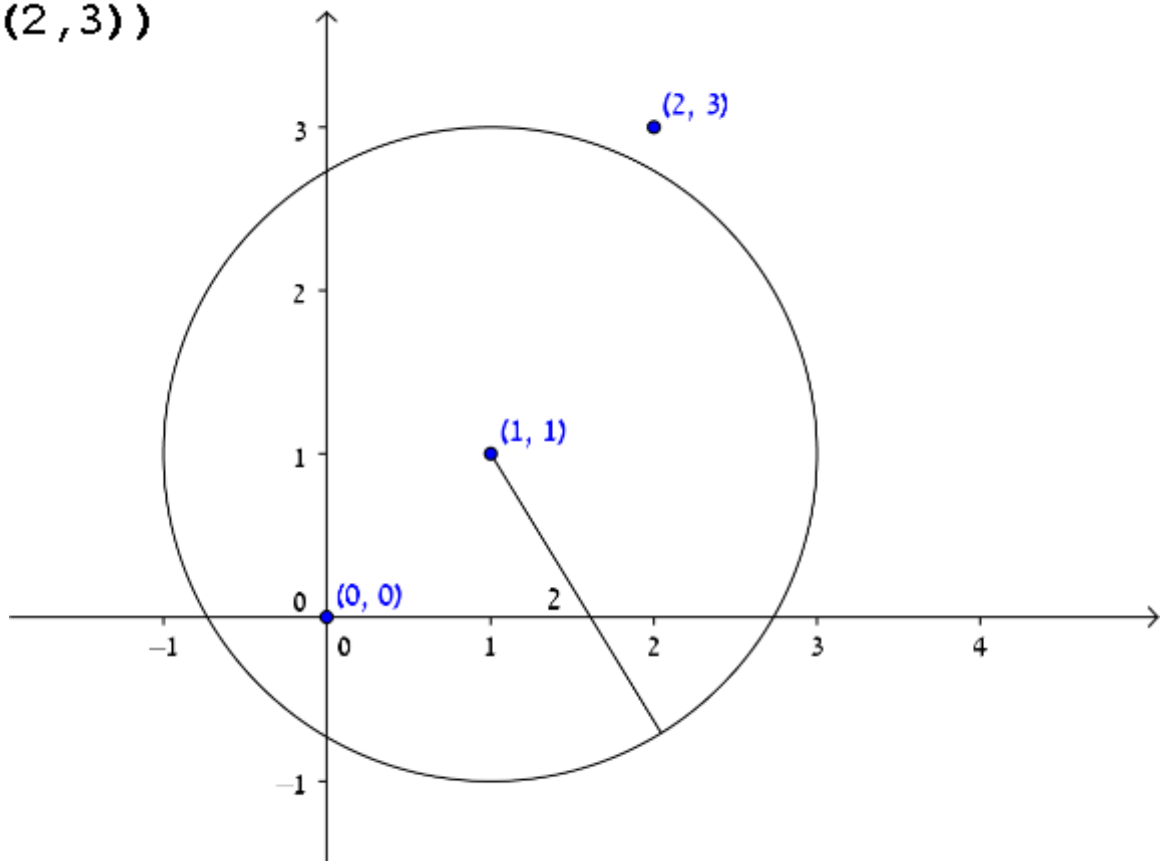
אוניברסיטת אריאל בשומרון

```
from math import sqrt

class Point:
    """represents a point in a 2D space.
    Attributes: x, y """
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def print_point(self):
        print('<', self.x, ',', self.y, '>')

    def distance(self, other):
        return sqrt((self.x-other.x)**2 + (self.y-other.y)**2)


class Circle:
    """ represents a circle shape
    attributes: center, radius"""
    def __init__(self, center, radius):
        self.center = center
        self.radius = radius

    def print_circle(self):
        print('center:', self.center.x, self.center.y,', radius:', self.radius)

    def in_circle(self, point):
        return self.center.distance(point) < self.radius
```

Usage example

אוניברסיטת אריאל בשומרון

```
>>> c = Circle(Point(1,1), 2)
>>> c.in_circle(Point(0,0))
True
>>> c.in_circle(Point(2,3))
False
```



Polymorphism

Enables working uniformly with objects of different types.

- Executing the same method on objects of different types will invoke the right version of the method !

- Assume you have a list that contains objects of different types, but all types implement a method named m().
- If you run over the list and invoke m() on each object, Python will execute the version of m appearing in the class defining the type of the current object.

Polymorphism - Example

אוניברסיטת אריאל בשומרון

```
class Cat:
    def talk(self):
        return 'Meow!'
class Dog:
    def talk(self):
       return 'Woof! Woof!’
class Duck:
    def talk(self):
        return ‘quack’

animals = [Cat(), Dog(), Cat(),Duck()]
for anim in animals:
    print anim.talk()
```

```
Meow!
Woof! Woof!
Meow!
```

Histogram (polymorphism)


```
def histogram(iterable):
    hist = {}
    for elem in iterable:
        hist[elem] = hist.get(elem, 0) + 1
    return hist
```

This function works for lists, tuples, strings and dictionaries if the elements of *iterable* can be used as keys in hist

```
>>> words = ['spam', 'egg', 'spam', 'bacon', 'spam']
>>> histogram(words)
{'bacon': 1, 'egg': 1, 'spam': 3}
```

Today

אוניברסיטת אריאל בשומרון

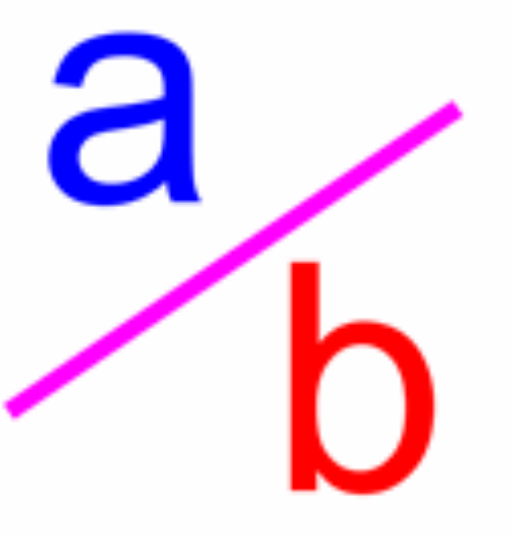
- We'll see a full example for introducing a new data type into Python, a data type which represents Rational numbers.
- We'll provide the new class with various methods that will enable using it naturally in python like predefined built-in types.

Today

אוניברסיטת אריאל בשומרון

- Rational numbers implementation using OOP
  - We will write a class that represents a rational number.
  - Using the new type should feel like native language types

1/1	1/2	1/3	1/4	1/5	1/6	1/7	1/8	1/9	…
2/1	2/2	2/3	2/4	2/5	2/6	2/7	2/8	2/9	…
3/1	3/2	3/3	3/4	3/5	3/6	3/7	3/8	3/9	…
4/1	4/2	4/3	4/4	4/5	4/6	4/7	4/8	4/9	…
5/1	5/2	5/3	5/4	5/5	5/6	5/7	5/8	5/9	…
6/1	6/2	6/3	6/4	6/5	6/6	6/7	6/8	6/9	…
7/1	7/2	7/3	7/4	7/5	7/6	7/7	7/8	7/9	…
8/1	8/2	8/3	8/4	8/5	8/6	8/7	8/8	8/9	…
9/1	9/2	9/3	9/4	9/5	9/6	9/7	9/8	9/9	…
…	…	…	…	…	…	…	…	…	…



Rational Numbers

אוניברסיטת אריאל בשומרון

- A rational number is a number that can be expressed as a ratio n/d (n, d integers, d not 0)
- Examples: 1/2, 2/3, 112/239, 2/1

1/1	1/2	1/3	1/4	1/5	1/6	1/7	1/8	1/9	…
2/1	2/2	2/3	2/4	2/5	2/6	2/7	2/8	2/9	…
3/1	3/2	3/3	3/4	3/5	3/6	3/7	3/8	3/9	…
4/1	4/2	4/3	4/4	4/5	4/6	4/7	4/8	4/9	…
5/1	5/2	5/3	5/4	5/5	5/6	5/7	5/8	5/9	…
6/1	6/2	6/3	6/4	6/5	6/6	6/7	6/8	6/9	…
7/1	7/2	7/3	7/4	7/5	7/6	7/7	7/8	7/9	…
8/1	8/2	8/3	8/4	8/5	8/6	8/7	8/8	8/9	…
9/1	9/2	9/3	9/4	9/5	9/6	9/7	9/8	9/9	…
…	…	…	…	…	…	…	…	…	…

Rational Class – specification

- **print** should work smoothly
- Support add, subtract, multiply, divide
- Immutable
- Should feel like native language types

```
>>> half = Rational(1,2)
>>> two_thirds = Rational(2,3)
>>> half / (2 + two_thirds)
3/16
>>> sum([1, Rational(3,2), 2])
9/2
```

Constructing a Rational

- What are the attributes?
- How should a client programmer create a new Rational object?

```
class Rational:
        """represents a rational number
        attributes: n,d"""
        def __init__(self,n,d):
                """n: numerator
                d: denumerator"""
                self.numer = n
                self.denom = d
```

Default Arguments to Constructor

- Constructors other then the primary?
- Example: a rational number with a denominator of 1 (e.g., 5/1 → 5)
- We would like to enable: **`Rational(5)`**

```
>>> x = Rational(5)

Traceback (most recent call last):
  File "<pyshell#2>", line 1, in <module>
    x = Rational(5)
TypeError: __init__() takes exactly 3 arguments (2 given)
```

Default Arguments to Constructor

אוניברסיטת אריאל בשומרון

```
def __init__(self,n,d=1):
        """n: numerator
        d: denumerator"""
        self.numer = n
        self.denom = d
```

```
>>> p = Rational(5) # == Rational(5,1)
>>> q = Rational(5, 2)
```

Checking Preconditions


```
>>> q = Rational(5,0)
>>> q   print support is not implemented yet…
5/0
```

Ensure the arguments are **valid** when the object is initialized:

```
def __init__(self, n, d=1):
        """n: numerator
        d: denominator"""
        if d == 0:
                print "denominator is set to 1, cannot be zero"
                d = 1
        self.numer = n
        self.denom = d
```

Checking Preconditions


```
>>> p = Rational(5,0)
Denominator is set to 1, cannot be zero!
>>> p
5/1
```

print support is not implemented yet…

Raising and exception is a better approach !

Rational: Raising an exception in case of invalid input

אוניברסיטת אריאל בשומרון

```
def __init__(self, n, d=1):
        """n: numerator
        d: denominator"""
        if d == 0:
                raise ZeroDivisionError("Denominator cannot be zero")

        self.numer = n
        self.denom = d
```

```
>>> r1 = Rational(3,0)

Traceback (most recent call last):
  File "<pyshell#4>", line 1, in <module>
    r1 = Rational(3,0)
  File "W:\teaching\python\8_code\rational.py", line 10, in __init__
    raise ZeroDivisionError("Denominator cannot be zero!")
ZeroDivisionError: Denominator cannot be zero!
```

Printing a Rational

אוניברסיטת אריאל בשומרון

```
>>> q = Rational(2,3)
>>> q.numer
2
>>> q.denom
3
>>> q
<__main__.Rational instance at
0x00000000003123BD32>
>>> print (q)
<__main__.Rational instance at
0x00000000003123BD32>
```

Implementing __repr__

__repr__ method:

returns the official **string representation** of an object

In class Rational:

```
def __repr__(self):
        return str(self.numer) + '/' + str(self.denom)
```

In the shell:

```
>>> q = Rational(2,3)
>>> q
2/3
>>> print(q)
2/3
```

Implementing __str__

- *__str__* method returns a **readable** string representation of an object
- The default implementation of *__str__* returns the return value of *__repr__*.
- Read here about the difference between *__str__* and *__repr__*

`__str__`

If you print an object, or pass it to `format`, `str.format`, or `str`, then if a `__str__` method is defined, that method will be called, otherwise, `__repr__` will be used.

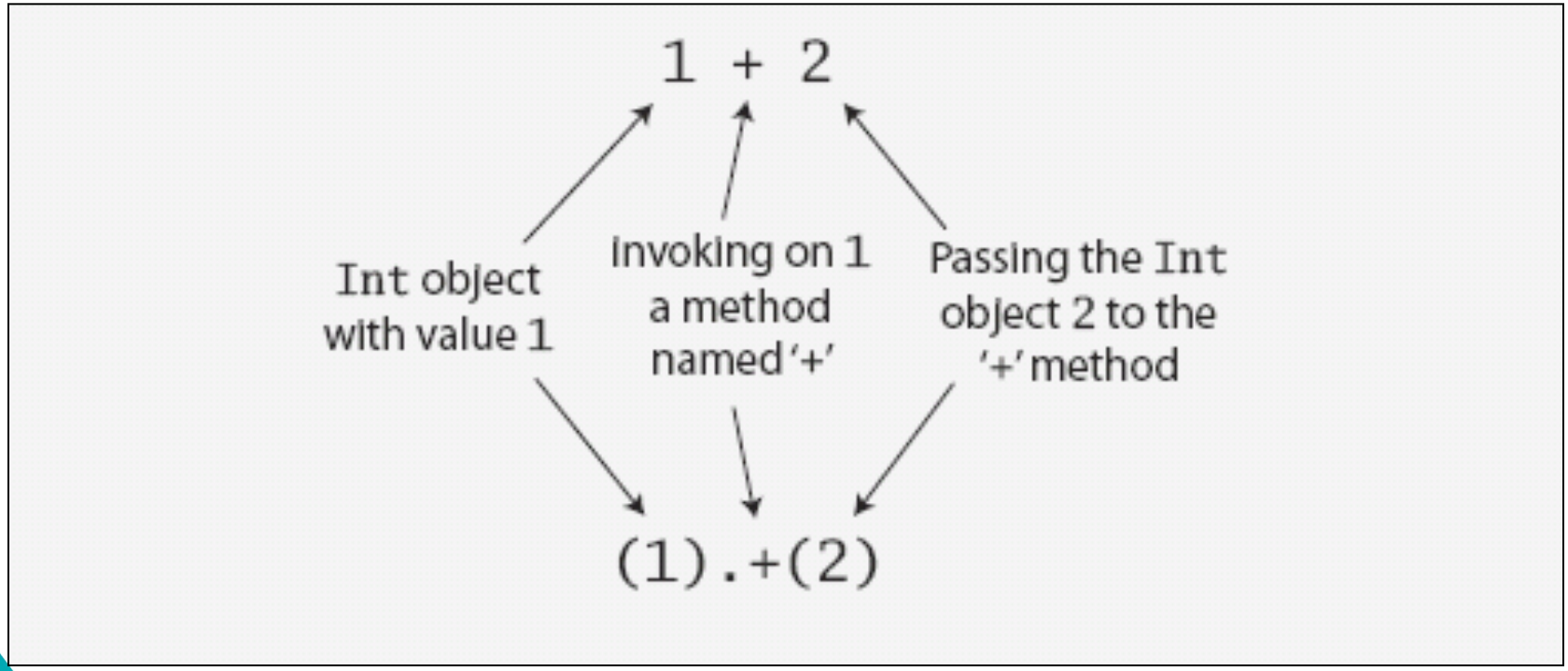
`__repr__`

The `__repr__` method is called by the builtin function `repr` and is what is echoed on your python shell when it evaluates an expression that returns an object.

Since it provides a backup for `__str__`, if you can only write one, start with `__repr__`

Defining Operators

- Why not use natural arithmetic operators?
- Operator precedence will be kept
- All operations are method calls



26

Operator Overloading

אוניברסיטת אריאל בשומרון

- By defining some **special methods**, you can specify the behavior of operators on user defined types

  +, -, *, /, <, >, ==, …

Adding Rational Numbers


```
>>> p = Rational(2,3)
>>> q = Rational(1,2)
>>> p+q

Traceback (most recent call last):
  File "<pyshell#49>", line 1, in <module>
    p+q
TypeError: unsupported operand type(s) for +: 'instance' and 'instance'
```

Arithmetic operators Overloading

```
>>> q = Rational(1,2)
>>> p = Rational(2,3)
```

Operator	Method
>>> p+q	__add__(self,other)
>>> -p	__neg__(self)
>>> p-q	__sub__(self,other)
>>> p*q	__mul__(self,other)
>>> p/q	__div__(self,other)

Define __add__ Method

$$\frac{a}{b}+\frac{c}{d}=\frac{a*d+c*b}{b*d}$$

```
def __add__(self,other):
        return Rational(self.numer * other.denom +
                        other.numer * self.denom,
                        self.denom * other.denom)
```

Shell:

```
>>> q = Rational(1,2)
>>> r = Rational(2,3)
>>> q+r
7/6
```

Other Arithmetic Operations Implementation

אוניברסיטת אריאל בשומרון

```
def __neg__(self):
        return Rational(-self.numer, self.denom)

def __sub__(self,other):
        return self + (-other)

def __mul__(self,other):
        return Rational(self.numer * other.numer,
                        self.denom * other.denom)

def __div__(self,other):
        return Rational(self.numer * other.denom,
                        self.denom * other.numer)
```

Other Arithmetic Operations

אוניברסיטת אריאל בשומרון

```
>>> q = Rational(1,2)
>>> p = Rational(2,3)
>>> -q
-1/2
>>> q-p
-1/6
>>> q*p
2/6
>>> q/p
3/4
>>> q*(q+p)
7/12
>>> q*p+q
10/12
```

What about p+2?

Mixed Arithmetic

- Now we can add and multiply rational numbers!
- What about mixed arithmetic: Rational+int or int+Rational?

```
>>> p = Rational(2,3)
>>> p + 2

Traceback (most recent call last):
  File "<pyshell#1>", line 1, in <module>
    p + 2
  File "C:\Python27\test-operators.py", line 15, in __add__
    return Rational(self.numer * other.denom + other.numer * self.denom, self.de
nom * other.denom)
AttributeError: 'int' object has no attribute 'denom'
>>> 2 + p

Traceback (most recent call last):
  File "<pyshell#2>", line 1, in <module>
    2 + p
TypeError: unsupported operand type(s) for +: 'int' and 'instance'
```

Mixed Arithmetic's

Let's try Rational + int first:

- first attempt:

  Rational + int => Rational + Rational(int)

```
>>> p = Rational(2,3)
>>> p + Rational(2)
8/3
```

  Works but not elegant or comfortable

- So…
- Add new methods for mixed addition and multiplication
- Will work thanks to polymorphism

Mixed Arithmetic's

Intuition: converting int to Rational is a good idea, we just need to figure out where to perform the conversion.

Inside __add__ method!

```
def __add__(self,other):
        other = Rational(other)
        return Rational(self.numer * other.denom
                        + other.numer * self.denom,
                        self.denom * other.denom)
```

Will this work?

Mixed Arithmetic's

אוניברסיטת אריאל בשומרון

```
>>> p = Rational(2,3)
>>> p + 2
8/3
```

```
>>> p = Rational(2,3)
>>> p+p

Traceback (most recent call last):
  File "<pyshell#3>", line 1, in <module>
    p+p
```

What happened?

Inside __add__ we convert *other* to Rational – but *other* is already a Rational.

Invoking Rational constructor with a Rational argument generates an unexpected object:

```
>>> Rational(p)
2/3/1
```

Solution: convert only for *other* of type int!

Revised __add__

- The built-in function `isinstance` takes a value and a class object, and returns True if the value is an instance of the class (False otherwise)
- **We only convert to Rational if the argument is of type int!**

```
def __add__(self,other):
        if isinstance(other, int):
                other = Rational(other)
        return Rational(self.numer * other.denom
                        + other.numer * self.denom,
                        self.denom * other.denom)
```

Revised __add__


```
>>> p = Rational(2,3)
>>> p+2
8/3
>>> p+p
12/9
```

What about p+2?

If we want Rational to be used just like any other numeric data type in respect to arithmetic operations, we must implement this behavior for **every operator** and **every data type** that is supported.

Implicit Conversions

אוניברסיטת אריאל בשומרון

- Now let's make int + Rational work:
- When we perform 2+p, __add__method that is called belongs to int, since and argument of type int appears on the left from the operator.
  - The problem: int class contains no __add__ method that takes a Rational argument ☹
- Intuition: we don't wish to change the implementation of int, the code should appear in Rational class.
- Is there a method that is invoked when Rational is on the **right** side of the operator?

__radd__ - Right Side Add

- __radd__ is invoked when a Rational object appears on the right side of the + operator

```
def __radd__(self,other):
        return self + other
```

Rational + int: we already know how to do it!

```
>>> p = Rational(2,3)
>>> 2+p
8/3
```

Implementating __radd__ using __add__ is only possible because addition of numbers is commutative: **a+b = b+a**

Rational addition

- We now support both int + Rational and Rational + int, so we may execute complex arithmetic operations and use built-in functions:

```
>>> 1 + Rational(1,2) + Rational(3,4) + 3
42/8
>>> sum([1, Rational(1,2), Rational(3,4), 3])
42/8
```

Comparisons

אוניברסיטת אריאל בשומרון

```
>>> x = Rational(1,2)
>>> y = Rational(1,3)
>>> x<y
True
>>> y<x
False
```

- Default implementation for these operators is based on the object addresses

What methods are responsible for comparison?

Operator	Method
>>>x == y	__eq__(self,other)
>>>x < y	__lt__(self,other)
>>>x <= y	__le__(self,other)
>>>x > y	__gt__(self,other)
>>>p >= q	__ge__(self,other)
>>>x != y	__ne__(self,other)

Comparisons

אוניברסיטת אריאל בשומרון

```
def __eq__(self,other):
        return self.numer == other.numer and self.denom == other.denom

def __lt__(self,other):
        return self.numer * other.denom < self.denom * other.numer

def __le__(self,other):
        return self.numer * other.denom <= self.denom * other.numer

def __gt__(self,other):
        return self.numer * other.denom > self.denom * other.numer

def __ge__(self,other):
        return self.numer * other.denom >= self.denom * other.numer
```

Comparisons


```
>>> x = Rational(1,2)
>>> y = Rational(1,3)
>>> x > y
True
>>> x < y
False
>>> x >= y
True
>>> x <= y
False
>>> max(x,y)
1/2
```

does max works?

Internal representation

אוניברסיטת אריאל בשומרון

```
>>> Rational(8,6) == Rational(4,3)
False
```

- But 8/6 is equal to 4/3!
- We need to normalize the Rational numbers to be able to compare them properly.
- To normalize we divide the numerator and denominator by their *greatest common divisor* (*gcd*)
- gcd(8,6) = 2 ➔ (8/2)/2 = 4/3

No need for Rational clients to be aware of this

Revised Rational


```
def __init__(self, n, d=1):
        """n: numerator
        d: denominator"""
        if d == 0:
                raise ZeroDivisionError("Denominator cannot be zero")
        gcd = fractions.gcd(n, d)
        self.numer = n/gcd
        self.denom = d/gcd
```

```
>>> Rational(8,6) == Rational(4,3)
True
```

The change in the constructor influences more than Rationals comparison:

```
>>> Rational(8,6)
4/3
```

How can we maintain both the un-normalized representation and the correct comparison?

Summary

אוניברסיטת אריאל בשומרון

- **We implemented a full class that naturally represents a Rational number**
  - Attributes, methods, constructor
  - Method overriding
  - Encapsulation
  - Define operators as method
  - Method overloading
  - Polymorphism
  - Automatic type conversion

Rational Numbers in Python

אוניברסיטת אריאל בשומרון

- There is a Python implementation of Rational numbers
- It is called fractions **http://docs.python.org/library/fractions.html**

The most awesome Beatle


```
class Beatle:
    def __init__(self, name, popularity, awesomeness):
        self.name = name
        self.popularity = popularity
        self.awesomeness = awesomeness
    def __repr__(self):
        return self.name+str((self.popularity, self.awesomeness))

beatles = []
beatles.append(Beatle("John", 10, 7))
beatles.append(Beatle("Paul", 9, 8))
beatles.append(Beatle("George", 7, 10))
beatles.append(Beatle("Ringo", 5, 5))

>>> beatles
[John(10, 7), Paul(9, 8), George(7, 10), Ringo(5, 5)]
```

How can we find out who is the most popular Beatle and who is the most awesome one?

The Beatle class

- Easy! We can add __lt__ implemention and use max.
- Is it good enough?
- No, when we implement __lt__ we'll have to choose our comparison criterion: either popularity or awesomeness

key argument for max

- max(*iterable*, [*key*])

key – an optional argument for max function. If specified, *key* is applied on each element of *iterable* .

max returns the element elem s.t. key(elem) is the maximum value among the other elements in *iterable*.

key argument for max

אוניברסיטת אריאל בשומרון

```
>>> lst = ["abc", "c", "bb"]
>>> max(lst)
'c'
>>> max(lst, key=len)
'abc'

>>> int_lst = [1,12,3]
>>> max(int_lst, key=len)

Traceback (most recent call last):
  File "<pyshell#15>", line 1, in <module>
    max(int_lst, key=len)
TypeError: object of type 'int' has no len()
```

We must be able to apply **key** on each **element** of the list

Who is the most awesome Beatle?

we can implement functions that will help us to compare Beatles according to different criteria

```
def popularity_key(beatle):
    return beatle.popularity

def awesomeness_key(beatle):
    return beatle.awesomeness
```

```
>>> max(beatles, key=popularity_key)
John(10, 7)
>>> max(beatles, key=awesomeness_key)
George(7, 10)
```

The most awesome Beatle

The most popular Beatle

Also works for min and sort!


```
>>> lst = ["abc", "c", "bb"]
>>> min(lst, key=len)
'c'
>>> lst.sort(key=len)
>>> lst
['c', 'bb', 'abc']
>>> min(beatles, key=awesomeness_key)
Ringo(5, 5)
>>> beatles.sort(key=awesomeness_key)
>>> beatles
[Ringo(5, 5), John(10, 7), Paul(9, 8), George(7, 10)]
```

Example: a Student class

אוניברסיטת אריאל בשומרון

```
class Student:
    def __init__(self,name,id,grade_list):
        self.name = name
        self.id=id
        self.grade_list=grade_list

    def calc_grade_average(self):
        s=0
        for g in self.grade_list:
            s+=g
        return s/len(self.grade_list)

    def __repr__(self):
        return self.name + ': ID=' + str(self.id) + ' Grades: ' + str(self.grade_list)

student1 = Student('Moshe',893715,[90, 93,81])
student2 = Student('David',608028,[78, 72,99])
print(student1.name,':',student1.calc_grade_average())
print(student2.name,':',student2.calc_grade_average())

students=[]
students.append(student1)
students.append(student2)

print(sorted(students, key = lambda x : x.calc_grade_average())) # prints the list of students sorted by student grade average
```

```
88.0
[Moshe: ID=893715 Grades: [90, 93, 81],
David: ID=608028 Grades: [78, 72, 99]]
```